THETIS SDR DOCUMENTATION

TCI Thetis Protocol

Ver. 1.10

Series: Thetis SDR TCI interoperability supplement referencing TCI Protocol Ver. 2.0

13 June 2026

Reference Notice

Scope

This document supplements the public **Expert Electronics TCI Protocol Ver. 2.0** document dated **12 January 2024**, published at https://github.com/ExpertSDR3/TCI. It does not replace the original specification. The official Expert Electronics document remains the base reference for the generic TCI grammar, transport model, and command families.

Purpose

This supplement records the current Thetis interpretation of the protocol, including Thetis mappings for selected standard TCI 2.0 commands, Thetis behaviour differences, and Thetis custom commands that are not part of the official Expert Electronics 2.0 document.

Audience

This document is intended for client developers, integrators, automation authors, and maintainers working against the Thetis TCI server.

Reference And Document Control

This supplement documents Thetis behaviour against the official Expert Electronics 2.0 protocol PDF.

Base specification	Expert Electronics, *TCI Protocol*, Ver. 2.0, 12 January 2024
Supplement title	TCI Thetis Protocol
Implementation basis	Thetis TCI server behaviour as of 13 June 2026, reconciled with the uploaded Expert Electronics TCI Protocol Ver. 2.0 text, current TCIServer.cs, and current clsTCIpermissions.cs
Coverage	Selected TCI 2.0 differences, WebSocket and secure WebSocket startup, authentication and permissions, standard command reconciliation, Thetis extension messages, current parsed command coverage, audio negotiation, normal and compact sensor reports, round trip timing echo, and compact display streaming

Version	Comment	Date
1.10	Added secure WebSocket setup, authentication handshake messages, permission roles, command permission mapping, parsed command coverage, and Thetis extension command notes for `rx_snb_enable`, `duplex`, `tune_twotone`, and `ps-a`.	13 June 2026
1.09	Updated against the 4 June 2026 `TCIServer.cs`. Added the per receiver/channel `rx_sensors_enable_ex` control and the compact `sr` sensor report, clarified TX sensor fields, added the missing spot maintenance/click notification messages implemented by the current server, and documented the Thetis `rt` round trip echo command.	4 June 2026
1.8	Updated against the current `TCIServer.cs`, `clsDisplayCompression.cs`, and `clsSpectrumProcessor.cs` files. Reworked display stream documentation for the compact `TD` binary payload, the strict 11-field `display_start` command, viewport controls, block delta compression, adaptive block selection, keyframe interval, accuracy mode, and display quantisation depth. Added the `audio_reconfigure_begin` and `audio_reconfigure_end` controls.	3 June 2026
1.7	Updated against the then current `TCIServer.cs` display streaming implementation. Added documentation for `display_delta_mode` and the older display delta binary description.	28 May 2026
1.6	Updated against the then current `TCIServer.cs`, `cmaster.cs`, and TCI RX audio path implementation. Added display stream control coverage, documented `audio_codec` plus the full `audio_opus_*` negotiation family, and expanded examples for stream settings and limit behaviour.	27 May 2026
1.5	Revalidated against the current `TCIServer.cs` implementation and the local Expert Electronics Protocol 2.0 text. Confirmed that the custom message section lists Thetis extensions only; base protocol commands remain documented only where Thetis behaviour differs.	26 April 2026
1.4	Updated to match the latest `TCIServer.cs` implementation. Added documentation for `tx_filter_band_ex` and `run_cat_ex`, and rolled the implementation basis forward to the current source.	20 April 2026
1.3	Updated to match the latest `TCIServer.cs` additions. Added documentation for `vfo_sync_ex`, `vfo_swap_ex`, `fm_deviation_ex`, and `agc_auto_ex`.	15 April 2026
1.2	Updated to match the current `TCIServer.cs` implementation. Documented the move from `tx_frequency_thetis` to `tx_frequency_ex`, added receiver extension commands for NB, attenuation, preamp, and CTUN, and added sensor plus stream negotiation notes for the Thetis TCI server.	14 April 2026
1.1	Updated the CW terminal family notes to reflect observed Protocol 2.0 command usage validated against ExpertSDR 3 v1.1.7, including receiver qualified `cw_terminal` and `cw_macros_empty` forms.	28 March 2026
1.0	Initial Thetis supplement for the Expert Electronics TCI Protocol Ver. 2.0 reference set. Focused on Thetis command differences, CW/CWX behaviour, routing notes, and Thetis custom commands.	26 March 2026

Contents

1 Introduction

1.1 Document Scope

1.2 Relationship To The Official 2.0 Protocol

2 Thetis Implementation, Connection, Security, And Permissions Overview

2.1 Transport, WebSocket, And WSS

2.2 Authentication Handshake

2.3 Permissions, Roles, And Startup State

3 Selected TCI 2.0 Commands With Thetis Behaviour Differences

3.1 Receiver, Audio, And AGC Controls

3.2 Locking, Noise Processing, And Extended Receiver State

3.3 Display Stream Control

3.4 Compact Display Binary Payload

3.5 Sensors, Audio Negotiation, And Opus Codec Settings

3.6 CW And KEYER Support Via CWX

4 Standard Reconciliation And Thetis Extensions

4.4 Current Parsed Command And Permission Map

5 Compatibility Notes

1 Introduction

1.1 Document Scope

This document covers the parts of TCI 2.0 where Thetis has implementation detail that matters to clients. It focuses on Thetis differences from the base protocol and on Thetis messages that fall outside the official Expert Electronics 2.0 specification.

1.2 Relationship To The Official 2.0 Protocol

The official Expert Electronics document remains the baseline command grammar reference. This supplement records Thetis behaviour when those commands are used against the Thetis TCI server. In the CW terminal family, the published Protocol 2.0 document differs from observed command usage in ExpertSDR 3, so the Thetis notes for those commands have been validated against ExpertSDR 3 v1.1.7.

> **Reading Rule**
>
> If a command is defined in the official 2.0 document and not called out here, clients should assume the official Expert Electronics behaviour model first. If a command is called out in this supplement, the Thetis note in this document takes precedence for Thetis interoperability.

2 Thetis Implementation, Connection, Security, And Permissions Overview

Thetis presents itself as a TCI 2.0 style server, but several command families map onto existing Thetis subsystems rather than separate internal protocol objects.

Base reference	Expert Electronics TCI Protocol Ver. 2.0, 12 January 2024
CW engine	CWX is used as the execution engine for TCI CW operations
AGC mapping	TCI AGC names are mapped onto Thetis AGC modes
Pan/balance	`rx_balance` maps onto Thetis pan controls
Notch filtering	`rx_nf_enable` maps to MNF, not MNF+
Lock model	`lock` and `vfo_lock` are adapted to Thetis VFO A / VFO B routing
TX frequency extension	Thetis publishes standard `tx_frequency` plus Thetis `tx_frequency_ex`
TX filter extension	Thetis publishes `tx_filter_band_ex` for transmit filter low/high edges using a normalised positive width range
Receiver extensions	Thetis publishes `rx_nb_enable_ex`, `rx_nr_enable_ex`, `rx_step_att_enabled_ex`, `rx_step_att_ex`, `rx_preamp_att_ex`, `rx_ctun_ex`, `fm_deviation_ex`, and `agc_auto_ex`
Sensors	Optional RX/TX sensor reports are timer driven and enabled by client request. The current server supports both normal RX reports and the compact per receiver/channel `sr` report enabled by `rx_sensors_enable_ex`
Audio streaming	RX/TX audio negotiation includes codec, sample type, channel count, packet size, TX buffering, and optional deferred reconfiguration
Display streaming	Thetis display output is published as compact WebSocket binary payloads beginning with the `TD` magic bytes and version `1`
Display compression	Display output supports raw, delta, block delta, and use previous frame payload kinds, with per receiver controls
VFO helpers	Thetis adds explicit VFO sync and swap helpers through `vfo_sync_ex` and `vfo_swap_ex`
Spot metadata	Thetis accepts structured metadata in the `spot` arg5 JSON payload
CAT passthrough	Thetis adds `run_cat_ex` to execute a single CAT command and return the CAT response in a TCI message
Round trip check	Thetis adds `rt` as a minimal echo command for client round trip timing checks

2.1 Transport, WebSocket, And WSS

Thetis uses a WebSocket transport for TCI text commands and binary stream frames. The default listener is `127.0.0.1:31001`. Clients normally connect with `ws://<host>:31001/`, then exchange normal TCI text messages after the HTTP WebSocket upgrade completes.

Item	Current Thetis behaviour
Plain WebSocket	The normal TCI listener accepts WebSocket upgrade requests on the configured TCI address and port. Text frames are UTF-8. Binary frames are used for IQ, audio, TX audio, and compact display payloads.
Secure WebSocket	When a certificate path is configured and the certificate contains a private key, Thetis also starts a secure listener on the same address and on `TCI port + 1`. With the default port, the secure endpoint is therefore `wss://<host>:31002/`.
Certificate requirements	The WSS listener is not created unless the certificate file exists and can be loaded as an `X509Certificate2` with a private key. If WSS setup fails, the plain WS listener can still run.

Item	Current Thetis behaviour
Keepalive	The server sends WebSocket ping frames containing `Thetis` approximately every 20 seconds. Clients should reply with normal WebSocket pong handling.

2.2 Authentication Handshake

Authentication is optional and can be enabled separately for WS and WSS. When authentication is required, Thetis does not send normal initialisation data immediately after the WebSocket upgrade. It first sends an authentication request and waits for the client access code.

Message	Direction	Meaning	Example
AUTHENTICATION_REQUIRED	Server to client	Authentication is required before any normal TCI state is sent. The same message is repeated if a non authentication command is sent before login.	`authentication_required:true;`
AUTHENTICATION_PINCODE	Client to server	Submits the access code. The entire argument text after the colon is trimmed and checked against the configured users.	`authentication_pincode:123456;`
AUTHENTICATION_OK	Server to client	The access code matched an enabled user. This is followed by the permission list and then the normal initialisation sequence.	`authentication_ok;`
AUTHENTICATION_PERMISSIONS	Server to client	Comma separated effective permission tokens for the authenticated session. These are the role permissions plus any extra user permissions.	`authentication_permissions:view_status,view_spectrum;`
AUTHENTICATION_FAIL	Server to client	The code was not accepted. The argument is the remaining attempt count.	`authentication_fail:4;`
AUTHENTICATION_LOCKED	Server to client	The session or source address is locked after too many failures. The argument is the lockout time in seconds. Current lockout is 300 seconds after 5 failed attempts.	`authentication_locked:300;`
PERMISSION_DENIED	Server to client	A command was authenticated but not allowed. If the command has a permission mapping,	`permission_denied:change_frequency;`

Message	Direction	Meaning	Example
		Thetis returns the missing permission token; if not, it returns the command name.	

Authentication Flow For Clients

Connect to WS or WSS, wait for either normal initialisation or `authentication_required:true;`. If authentication is required, send `authentication_pincode:<access_code>;`. After `authentication_ok;` and `authentication_permissions:...;`, process the normal startup state ending with `ready;`. Do not assume that initialisation will be sent before successful authentication.

2.3 Permissions, Roles, And Startup State

Permissions are enforced only when authentication is enabled for the connection type being used. When authentication is disabled, the current parser accepts all implemented commands without permission checks.

Permission token	Purpose
`view_status`	Read radio status and low impact status helpers.
`view_spectrum`	Start or stop IQ and display/spectrum streams.
`listen_rx_audio`	Start or stop RX audio and line out streams.
`change_rx_audio`	Change audio sample settings and RX audio related controls.
`change_frequency`	Change frequency, IF, VFO, and related tuning controls.
`change_mode`	Change demodulation mode.
`change_rx_filter`	Change RX filter edges.
`change_rx_dsp`	Change RX DSP controls such as NR, NB, ANF, APF, notch, and AGC.
`change_rx_frontend`	Change RX enable, CTUN, attenuator, preamp, FM deviation, and VFO sync controls.
`change_display`	Change display stream settings such as pixels, zoom, pan, FFT, averaging, and compression.
`tx_ptt`	Control transmit PTT.
`tx_tune`	Control tune.
`tx_power`	Change drive and tune drive.
`tx_settings`	Change split, TX filter, and TX profile settings.
`tx_audio`	Send binary TX audio frames.
`cw_keyer`	Use CW macro, message, and keyer controls.
`spots`	Add, delete, clear, or manage spots.
`radio_power`	Start or stop the radio.
`admin`	Use administrative extensions such as shutdown and CAT passthrough.

Built in role	Permissions
Viewer	`view_status`, `view_spectrum`
Listener	Viewer plus `listen_rx_audio`

Built in role	Permissions
RX Operator	Listener plus `change_rx_audio`, `change_frequency`, `change_mode`, `change_rx_filter`, `change_rx_dsp`, and `change_display`
Advanced RX Operator	RX Operator plus `change_rx_frontend`
TX Operator	Advanced RX Operator plus `tx_ptt`, `tx_tune`, `tx_power`, `tx_settings`, and `cw_keyer`
Admin	All normal permissions, including `tx_audio`, `spots`, `radio_power`, and `admin`

Default Permission Configuration

The default configuration creates a `guest` user assigned to the Listener role with access code `123456`. Access codes are stored as PBKDF2 SHA256 hashes using 100000 iterations, 16 byte salts, and 32 byte hashes. A deployed system should replace the default access code before exposing authenticated TCI access.

Startup State After Authentication

After successful authentication, or immediately when authentication is disabled, Thetis sends the normal initialisation and state snapshot. The current startup begins with `protocol`, `device`, `receive_only`, `trx_count`, `channels_count`, `vfo_limits`, `if_limits`, and `modulations_list`, then sends the current radio state, audio settings, display related defaults, RX/TX enable state, lock state, sensors related state, TX profile/calibration extensions, and finally `ready;`.

3 Selected TCI 2.0 Commands With Thetis Behaviour Differences

This section lists standard 2.0 commands where Thetis behaviour, mapping, or restrictions differ in a way that client authors need to consider. Commands not listed here should be treated as normal base protocol behaviour.

3.1 Receiver, Audio, And AGC Controls

Command	Direction	Thetis behaviour	Example
VOLUME	Bidirectional	Controls Thetis master AF rather than receiver gain.	`volume:-12;`
RX_VOLUME	Bidirectional	Receiver and channel audio gain. Thetis uses the form `rx_volume:<rx>,<channel>,<db>;` and reports values in dB, not a raw linear percentage.	`rx_volume:0,0,-6.00;`
RX_BALANCE	Bidirectional	Maps to Thetis receiver pan. Thetis uses the form `rx_balance:<rx>,<channel>,<balance>;` with a range of `-40.0` to `40.0`.	`rx_balance:0,0,15.0;`
AGC_MODE	Bidirectional	Mapped to Thetis AGC modes. Thetis supports more AGC modes than the base TCI names expose.	`agc_mode:0,fast;`
RX_NB_ENABLE	Bidirectional	Controls the simple NB enabled state. The extended form `rx_nb_enable_ex` also carries the active NB algorithm index.	`rx_nb_enable:0,true;`
RX_NR_ENABLE	Bidirectional	Controls the simple NR enabled state. The extended form `rx_nr_enable_ex` also carries the active NR algorithm index.	`rx_nr_enable:0,true;`
RX_APF_ENABLE	Bidirectional	Applies to RX1 main and RX2 main only, and only in CW use.	`rx_apf_enable:1,true;`

AGC_MODE Mapping

`off`	FIXD
`normal`	MED
`fast`	FAST
`slow`	SLOW
`long`	LONG
`custom`	CUSTOM

Accepted aliases on input include `fixd`, `fixed`, `med`, and `medium`.

3.2 Locking, Noise Processing, And Extended Receiver State

Command	Direction	Thetis behaviour	Example
VFO_LOCK	Notification + control	Mapped onto Thetis VFO A / VFO B lock model. When RX2 is enabled, VFO B is represented on receiver 1 channels.	`vfo_lock:1,0,true;`
VFO_SYNC_EX	Bidirectional	Thetis extension for VFO sync state. Use the bare command to query, or a single boolean argument to enable or disable synchronisation.	`vfo_sync_ex:true;`
VFO_SWAP_EX	Unidirectional	Thetis extension that swaps the active VFO assignments immediately. This is a bare command with no arguments.	`vfo_swap_ex;`
RX_NR_ENABLE / RX_NR_ENABLE_EX	Bidirectional	`rx_nr_enable` exposes simple on/off NR state. `rx_nr_enable_ex` adds the active NR algorithm index. Get requests return both forms.	`rx_nr_enable_ex:0,true,2;`

Command	Direction	Thetis behaviour	Example
RX_NB_ENABLE / RX_NB_ENABLE_EX	Bidirectional	`rx_nb_enable` exposes simple on/off NB state. `rx_nb_enable_ex` adds the selected NB algorithm index.	`rx_nb_enable_ex:0,true,1;`
RX_NF_ENABLE	Bidirectional	Maps to Thetis manual notch, MNF, not MNF+ notch creation.	`rx_nf_enable:0,true;`
RX_STEP_ATT_ENABLED_EX	Bidirectional	Thetis extension for step attenuator enable state by receiver.	`rx_step_att_enabled_ex:0,true;`
RX_STEP_ATT_EX	Bidirectional	Thetis extension for step attenuator value in dB by receiver. This field defines attenuation directly, so values are positive dB attenuation amounts.	`rx_step_att_ex:0,10;`
RX_PREAMP_ATT_EX	Bidirectional	Thetis extension for preamp gain behaviour. This field is gain oriented; when the preamp path is being used as an attenuator, attenuation is expressed using negative values. Positive values are currently ignored by Thetis.	`rx_preamp_att_ex:0,-10;`
RX_CTUN_EX	Bidirectional	Thetis extension for CTUN state by receiver.	`rx_ctun_ex:0,true;`
FM_DEVIATION_EX	Bidirectional	Thetis extension for FM deviation selection by receiver query context. Current supported set values are `2500` and `5000` Hz.	`fm_deviation_ex:0,2500;`
AGC_AUTO_EX	Bidirectional	Thetis extension for AGC auto mode enable state by receiver.	`agc_auto_ex:0,true;`
TX_FILTER_BAND_EX	Bidirectional	Thetis extension for TX filter low/high edges. Query with no arguments, or set with low/high values in Hz. Thetis normalises values to non-negative low/high order and enforces a minimum 100 Hz span.	`tx_filter_band_ex:100,2900;`

VFO_LOCK Routing

When RX2 is disabled, `vfo_lock:0,0` refers to VFO A and `vfo_lock:0,1` refers to VFO B. When RX2 is enabled, `vfo_lock:0,0` stays on VFO A and VFO B is represented on receiver 1 using `vfo_lock:1,0` and `vfo_lock:1,1`.

Step Attenuator And Preamp Convention

- `rx_step_att_ex` defines attenuation in dB, so client values are positive attenuation amounts such as `10`, `20`, or `30`.
- `rx_preamp_att_ex` defines the amount of gain on the preamp path. When that path is being used as attenuation, the attenuation is represented by negative values such as `-10` or `-20`.
- Values above `0` are currently ignored for `rx_preamp_att_ex`.

3.3 Display Stream Control

Thetis implements a display stream control family that drives receiver spectrum subscriptions. Display output is sent as compact WebSocket binary payloads described in section 3.4.

Command	Direction	Thetis behaviour	Example
DISPLAY_START	Bidirectional	Starts or reconfigures a display stream subscription for receiver `0` or `1`. The current implementation requires the 11-field form `<rx>,<pixels>,<fps>,<fft>,<window>,<detector>,<average_mode>,<average_time_ms>,<one_hz>,<min_dbm>,<max_dbm>`.	`display_start:0,1024,15,8192,4,0,2,250.0,true,-140.0,-20.0;`
DISPLAY_STOP	Bidirectional	Stops the display stream subscription for the selected receiver. When the last subscription is removed, the internal spectrum processor is disposed for that TCI client.	`display_stop:0;`
DISPLAY_PIXELS	Bidirectional	Sets the display line width. Thetis accepts values in the range `64` through `32768`.	`display_pixels:0,2048;`
DISPLAY_ZOOM	Bidirectional	Sets zoom level from `0.0` to `1.0` and a zoom centre frequency in Hz. Thetis replies with the normalised zoom and also sends `display_pan` with the resulting pan value.	`display_zoom:0,0.650000,7074000;`
DISPLAY_PAN	Bidirectional	Sets pan position from `0.0` to `1.0`. While zoomed in, this clears the explicit zoom centre tracking state for the receiver.	`display_pan:0,0.500000;`
DISPLAY_DELTA_MODE	Control	Enables or disables delta capable display publication for the selected active display subscription. When disabled, block delta mode is also disabled.	`display_delta_mode:0,true;`
DISPLAY_BLOCK_DELTA_MODE	Control	Enables or disables block delta display publication. Enabling block delta also	`display_block_delta_mode:0,true;`

Command	Direction	Thetis behaviour	Example
		enables normal delta mode for that receiver.	
DISPLAY_ACCURACY_MODE	Control	Sets display compression tolerance. Values `0` through `4` are quantised byte thresholds. Value `5` enables an automatic threshold based on roughly 0.5 dB resolution.	`display_accuracy_mode:0,5;`
DISPLAY_ADAPTIVE_BLOCKS	Control	Enables adaptive block size selection for block delta payloads. Thetis chooses the smallest payload from block sizes `64`, `32`, `16`, and `128`.	`display_adaptive_blocks:0,true;`
DISPLAY_KEYFRAME_INTERVAL	Control	Sets the maximum number of delta frames between raw frames. Values are clamped to `0` through `1000000`. `0` disables the forced keyframe interval.	`display_keyframe_interval:0,30;`
DISPLAY_QUANT_BITS	Control	Sets the number of bits used to quantise raw display samples. Thetis clamps to `6`, `7`, or `8` bits.	`display_quant_bits:0,7;`
DISPLAY_FRAMERATE	Bidirectional	Sets the target display update rate in frames per second. Thetis accepts values in the range `1` through `120`.	`display_framerate:0,25;`
DISPLAY_FFTSIZE	Bidirectional	Sets the spectrum FFT size. Thetis works within `4096` through `262144` and normalises the request to the nearest power of two in that set.	`display_fftsize:0,16384;`
DISPLAY_WINDOW	Bidirectional	Sets the spectrum window type index. Thetis accepts values in the range `0` through `6`.	`display_window:0,4;`
DISPLAY_DETECTOR	Bidirectional	Sets the detector type index for the panadapter path. Thetis accepts values in the range `0` through `4`.	`display_detector:0,1;`

Command	Direction	Thetis behaviour	Example
DISPLAY_AVERAGING	Bidirectional	Sets the averaging mode index. Thetis accepts values in the range `0` through `3`.	`display_averaging:0,2;`
DISPLAY_AVERAGE_TIME	Bidirectional	Sets averaging time in milliseconds. Thetis works within `1.0` through `9999.0` ms.	`display_average_time:0,250.0;`
DISPLAY_ONE_HZ	Bidirectional	Enables or disables one hertz normalisation. Thetis accepts `true` / `false` and also `1` / `0`.	`display_one_hz:0,true;`
DISPLAY_DBM_RANGE	Bidirectional	Sets the quantisation range used when Thetis converts display dBm values into outbound display values. Each endpoint is limited to `-200.0` through `200.0` dBm. If `max` is not greater than `min`, Thetis applies a minimum 1 dB span.	`display_dbm_range:0,-140.0,-20.0;`

Display Setting Examples

- Start a compact RX1 feed: `display_start:0,1024,15,8192,4,0,0,120.0,false,-140.0,-20.0;`
- Enable delta and block delta compression: `display_delta_mode:0,true;` then `display_block_delta_mode:0,true;`
- Use adaptive blocks and 7 bit raw quantisation: `display_adaptive_blocks:0,true;` then `display_quant_bits:0,7;`
- Force a raw keyframe roughly every 30 display updates: `display_keyframe_interval:0,30;`
- Zoom to a frequency and then pan within the zoomed view: `display_zoom:0,0.650000,7074000;` then `display_pan:0,0.500000;`

Display Stream Notes

- Thetis exposes display subscriptions for receiver indexes `0` and `1` only.
- The current `display_start` handler requires exactly 11 arguments. The older 6-field form is no longer accepted by this implementation.
- Changing pixels, frame rate, FFT size, window, detector, averaging, average time, one hertz state, dBm range, zoom, pan, or display compression settings resets the saved prior frame, so the next update is sent as a raw compact display payload.
- When the display frequency span or left edge changes, Thetis resets display compression state and sends a raw compact display payload.
- All display compression controls apply to an existing active subscription. Start the display stream before setting compression controls.

3.4 Compact Display Binary Payload

Thetis display binary frames now use a compact display payload rather than the earlier 64-byte stream header used by audio and IQ streams. The payload starts with the ASCII magic bytes `T` and `D`, followed by a one byte version field.

Offset	Field	Description
`0`	Magic 0	`84`, ASCII `T`
`1`	Magic 1	`68`, ASCII `D`

Offset	Field	Description
`2`	Version	`1`
`3`	Payload kind	`0` = raw, `1` = delta, `2` = block delta, `3` = use previous frame
`4`	Receiver	Receiver index, normally `0` or `1`

Raw Payload Kind 0

Offset	Field	Description
`5`	Quantisation bits	`6`, `7`, or `8`
`6..9`	Hz left	Little endian unsigned 32 bit value for the left edge of the display span
`10..13`	Hz span	Little endian unsigned 32 bit value for the display span width
`14..17`	Logical pixel count	Little endian unsigned 32 bit value for the number of display points
`18..`	Sample payload	Raw quantised display samples. At 8 bits, this is one byte per pixel. At 6 or 7 bits, values are packed most significant bit first.

Delta Payload Kind 1

Offset	Field	Description
`5`	Delta bits	Bits used for each signed delta. Thetis uses `2` through `7` according to the largest per pixel change.
`6..`	Delta payload	Each value is packed most significant bit first and is decoded as `packed - ((1 << (bits - 1)) - 1)`, then added to the previous frame value.

Block Delta Payload Kind 2

Offset	Field	Description
`5`	Block size	Block size used for this payload. Default is `64`. With adaptive blocks enabled Thetis may use `64`, `32`, `16`, or `128`.
`6..`	Block mode table and data	The first `ceil(pixel_count / block_size)` bytes are block modes. Mode `0` means raw block bytes follow, mode `1` means unchanged block, and modes `2` through `7` mean a packed delta block using that number of delta bits.

Use Previous Frame Payload Kind 3

Kind `3` carries only the common 5 byte compact header. It tells the client to reuse the previous decoded display frame for that receiver. Thetis emits this kind only when delta mode is enabled and the new quantised display frame is unchanged after the active accuracy threshold is applied.

Client Decode Rules

- Expect the first frame after `display_start` to be raw because no previous frame exists.
- Store the latest decoded frame for each receiver. Delta, block delta, and use previous frame payloads are relative to that stored frame.
- Use the most recent raw payload metadata for pixel count, left edge, span, and quantisation depth until the next raw payload arrives.
- If the client loses any compact display binary frame, it should ignore later delta or block delta payloads for that receiver until the next raw payload arrives.

3.5 Sensors, Audio Negotiation, And Opus Codec Settings

The Thetis TCI server supports optional sensor polling plus negotiated RX/TX audio streaming. Current code supports both PCM audio transport and Opus codec negotiation.

Command	Direction	Thetis behaviour	Example
RX_SENSORS_ENABLE	Control	Enables or disables periodic RX sensor reports. Optional second argument sets the interval in milliseconds.	`rx_sensors_enable:true,200;`
RX_SENSORS_ENABLE_EX	Control	Enables or disables compact per receiver/channel RX sensor reports. Fields are receiver index, channel index, enabled state, and optional interval in milliseconds. This causes Thetis to publish the short `sr` report for the selected pair.	`rx_sensors_enable_ex:0,0,true,200;`
SR	Reply	Compact RX sensor report emitted by the extended RX sensor subscription. Fields are receiver index, channel index, signal dBm, average signal dBm, peak bin signal dBm, and noise floor dBm.	`sr:0,0,-71.5,-73.2,-69.8,-118.4;`
TX_SENSORS_ENABLE	Control	Enables or disables periodic TX sensor reports. Optional second argument sets the interval in milliseconds. Reports use `tx_sensors:<rx>,<mic_dbm>,<rms_watts>,<peak_watts>,<swr>;`.	`tx_sensors_enable:true,200;`
AUDIO_RECONFIGURE_BEGIN	Control	Begins a grouped audio reconfiguration. In the current parser this is handled by the colon form `audio_reconfigure_begin:;`; the bare `audio_reconfigure_begin;` form is not in the bare command switch. Audio compression reset and queued TX audio clearing are deferred until the matching end command unwinds the reconfiguration depth to zero.	`audio_reconfigure_begin:;`
AUDIO_RECONFIGURE_END	Control	Ends a grouped audio reconfiguration. In the current parser this is handled by the colon form `audio_reconfigure_end:;`; the bare `audio_reconfigure_end;` form is not in the bare command switch. When the nesting depth returns to zero, any deferred RX audio reset and TX audio queue clear are applied.	`audio_reconfigure_end:;`
AUDIO_SAMPLERATE	Bidirectional	Sets negotiated audio sample rate. In PCM mode Thetis accepts the standard TCI set `8000`, `12000`, `24000`, and `48000`. In Opus mode it also accepts `16000`.	`audio_samplerate:48000;`
AUDIO_CODEC	Bidirectional	Selects the negotiated audio codec family. Thetis accepts `pcm` and `opus`.	`audio_codec:opus;`
AUDIO_STREAM_SAMPLE_TYPE	Bidirectional	Selects TX/RX audio sample encoding. Thetis accepts `int16`, `int24`, `int32`, `float32`, and also `opus` as a shorthand that switches the codec family to Opus.	`audio_stream_sample_type:float32;`
AUDIO_STREAM_CHANNELS	Bidirectional	Selects audio stream channel count. Thetis accepts `1` or `2`.	`audio_stream_channels:2;`
AUDIO_STREAM_SAMPLES	Bidirectional	Selects the negotiated audio packet sample count. Thetis accepts values from `20` to `4096`. In Opus mode, only sizes that map to a valid Opus frame duration at the current sample rate are applied.	`audio_stream_samples:2048;`

Command	Direction	Thetis behaviour	Example
TX_STREAM_AUDIO_BUFFERING	Bidirectional	Controls TX audio buffering in milliseconds. Thetis accepts values from `50` to `500`.	`tx_stream_audio_buffering:50;`
AUDIO_OPUS_APPLICATION	Bidirectional	Sets Opus application mode. Thetis accepts `audio`, `voip`, and `lowdelay`.	`audio_opus_application:audio;`
AUDIO_OPUS_SIGNAL	Bidirectional	Sets Opus signal hint. Thetis accepts `auto`, `voice`, and `music`.	`audio_opus_signal:voice;`
AUDIO_OPUS_BANDWIDTH	Bidirectional	Sets the Opus bandwidth hint. Thetis accepts `auto`, `narrowband`, `mediumband`, `wideband`, `superwideband`, and `fullband`.	`audio_opus_bandwidth:fullband;`
AUDIO_OPUS_BITRATE	Bidirectional	Sets Opus target bitrate in bits per second. Thetis works within `6000` through `510000` bps.	`audio_opus_bitrate:64000;`
AUDIO_OPUS_VBR	Bidirectional	Enables or disables Opus VBR mode.	`audio_opus_vbr:true;`
AUDIO_OPUS_CVBR	Bidirectional	Enables or disables constrained VBR mode.	`audio_opus_cvbr:false;`
AUDIO_OPUS_COMPLEXITY	Bidirectional	Sets Opus encoder complexity. Thetis accepts values in the range `0` through `10`.	`audio_opus_complexity:10;`
AUDIO_OPUS_FEC	Bidirectional	Enables or disables in band forward error correction.	`audio_opus_fec:true;`
AUDIO_OPUS_DTX	Bidirectional	Enables or disables Opus DTX.	`audio_opus_dtx:false;`
AUDIO_OPUS_PACKET_LOSS	Bidirectional	Sets expected packet loss percentage for encoder tuning. Thetis accepts values in the range `0` through `100`.	`audio_opus_packet_loss:5;`
AUDIO_OPUS_FRAME_MS	Bidirectional	Sets Opus frame duration in milliseconds. Thetis normalises to `2.5`, `5`, `10`, `20`, `40`, or `60` ms, then updates `audio_stream_samples` to the matching sample count.	`audio_opus_frame_ms:20.0;`

Grouped Audio Reconfiguration

Clients can wrap several audio negotiation commands between `audio_reconfigure_begin:;` and `audio_reconfigure_end:;`. This lets Thetis defer RX audio compression reset and queued TX audio clearing until all related settings have been applied.

`audio_reconfigure_begin:;` can be nested. The deferred reset only occurs when matching `audio_reconfigure_end:;` calls reduce the depth back to zero.

Sensor Polling Behaviour

- Requested RX and TX sensor intervals are limited by Thetis to `30` through `1000` ms.
- When sensors are enabled, Thetis fires the first timer immediately, then continues at the negotiated interval.
- Normal RX reports are published as `rx_sensors`, `rx_channel_sensors`, and Thetis `rx_channel_sensors_ex`.
- Extended RX sensor subscriptions use `rx_sensors_enable_ex:<rx>,<channel>,<enabled>[,<interval_ms>];` and publish the compact `sr` report with signal, average signal, peak bin, and noise floor values.
- TX reports are published as `tx_sensors` with mic level dBm, RMS power watts, peak power watts, and SWR, and are currently mirrored on both receiver indexes for compatibility.

Audio Negotiation Defaults

- Thetis starts with `audio_codec:pcm;`, `audio_stream_sample_type:float32;`, `audio_stream_channels:2;`, and `tx_stream_audio_buffering:50;`.
- The default PCM audio sample rate is `48000`. The default sample count helper produces `256` at 8 kHz, `512` at 12 kHz, `1024` at 24 kHz, and `2048` at 48 kHz.

- In Opus mode, `16000` is also accepted and maps to a default sample count of `512`.
- When Opus is selected, Thetis publishes sample type `opus` and keeps `audio_stream_samples` aligned with the active Opus frame duration.

Frame Size Examples

- At `48000` Hz, `audio_opus_frame_ms:2.5;` maps to `audio_stream_samples:120;`.
- At `48000` Hz, `audio_opus_frame_ms:10.0;` maps to `audio_stream_samples:480;`.
- At `48000` Hz, `audio_opus_frame_ms:20.0;` maps to `audio_stream_samples:960;`.
- At `16000` Hz, `audio_opus_frame_ms:20.0;` maps to `audio_stream_samples:320;`.

3.6 CW And KEYER Support Via CWX

Thetis implements the TCI CW family using CWX as the execution engine. This keeps remote CW behaviour aligned with the native Thetis CWX subsystem rather than creating a second CW transmit path.

Command	Direction	Thetis behaviour	Example
CW_MACROS_SPEED	Bidirectional	Controls CWX macro speed.	`cw_macros_speed:30;`
CW_MACROS_DELAY	Bidirectional	Controls CWX PTT delay.	`cw_macros_delay:100;`
CW_KEYER_SPEED	Bidirectional	Controls Thetis keyer speed.	`cw_keyer_speed:24;`
CW_MACROS_SPEED_UP	Unidirectional	Raises CWX macro speed.	`cw_macros_speed_up:5;`
CW_MACROS_SPEED_DOWN	Unidirectional	Lowers CWX macro speed.	`cw_macros_speed_down:3;`
CW_MACROS	Bidirectional use	Sends CW text through CWX.	`cw_macros:0,cq cq de mw0lge;`
CW_TERMINAL	Bidirectional use	Enables or clears terminal mode in the Thetis CW controller. Thetis expects the receiver qualified form validated against ExpertSDR 3.	`cw_terminal:0,true;`
CW_MSG	Bidirectional use	Structured prefix / callsign / suffix send using CWX.	`cw_msg:0,tu,mw0lge,599 001;`
CW_MACROS_STOP	Unidirectional	Stops the active TCI CW owner and releases TX even if terminal mode remains enabled.	`cw_macros_stop;`
CW_MACROS_EMPTY	Notification	Sent in terminal mode when the last queued character starts transmitting. Thetis emits the receiver qualified form validated against ExpertSDR 3.	`cw_macros_empty:0;`
CALLSIGN_SEND	Notification	Sent when the callsign portion of `cw_msg` has been transmitted.	`callsign_send:mw0lge;`
KEYER	Unidirectional	Uses the CWX key path for timed key down / key up operation.	`keyer:0,true,0;`

Observed CW Protocol 2.0 Deviations

- The published TCI 2.0 PDF shows `cw_terminal:true;` and `cw_terminal:false;`, but ExpertSDR 3 v1.1.7 uses receiver qualified forms such as `cw_terminal:0,true;` and `cw_terminal:0,false;`.
- The published TCI 2.0 PDF shows `cw_macros_empty;`, but ExpertSDR 3 v1.1.7 emits `cw_macros_empty:0;`.
- Thetis follows the receiver qualified forms above for compatibility with observed Expert Electronics Protocol 2.0 behaviour.
- In terminal mode, Thetis remains in TX after the macro queue drains until an explicit `cw_terminal:<rx>,false;` or `cw_macros_stop;` is received.

KEYER Behaviour In Thetis

- `keyer` is implemented through CWX, not through the macro text queue.
- The selected TX side must already be in `cwl` or `cwu`.
- `keyer:0,...` selects TX on VFO A. If RX2 is enabled and TX is currently on VFO B, Thetis forces TX back to VFO A first.
- `keyer:1,...` only operates when RX2 is enabled and forces TX to VFO B before the CW mode check.

4 Standard Reconciliation And Thetis Extensions

This section separates official Expert Electronics TCI 2.0 commands from messages that are currently Thetis specific. The official protocol is still the base reference; this table records how the current Thetis server differs from, extends, or omits parts of that base.

4.1 Official 2.0 Commands That Are Not Thetis Custom Commands

The uploaded official TCI 2.0 text already defines several commands that were previously listed in this supplement as custom. They remain documented elsewhere only where Thetis behaviour differs.

Command	Official 2.0 status	Thetis note
SPOT_CLEAR	Standard	Official TCI 2.0 command for deleting all spots from the panorama. Thetis implements it through its spot manager.
SPOT_DELETE	Standard	Official TCI 2.0 command for deleting a single spot. Thetis accepts one argument and passes it to the spot manager delete path.
CLICKED_ON_SPOT	Standard notification	Official legacy spot click notification. Thetis emits `clicked_on_spot:<callsign>,<frequency_hz>;`.
RX_CLICKED_ON_SPOT	Standard notification	Official receiver/channel qualified spot click notification. Thetis emits receiver index, channel index, callsign, and frequency.
LINE_OUT_START / LINE_OUT_STOP	Standard	Official line out stream controls. Thetis maps the receiver argument onto its available line out / VAC controls when the setup form is available.
SET_IN_FOCUS	Standard	Official client to server focus request. Thetis brings the console form to the foreground.
VFO_LOCK	Standard in TCI 2.0	Thetis supports the receiver/channel form and maps it onto VFO A / VFO B locks, with RX2 routing rules documented in section 3.2.
RX_CHANNEL_SENSORS	Standard in TCI 2.0	Thetis emits the official three-field report and also emits the extended `rx_channel_sensors_ex` report when normal RX sensors are due.

4.2 Official 2.0 Commands Not Currently Parsed Or Emitted By Thetis

The current `TCIServer.cs` does not appear to implement the following official command families. Clients should not rely on them when targeting Thetis unless support is added later.

Official command	Thetis status in current source
RX_NB_PARAM	Not parsed as a client command in the current server. Thetis exposes NB state through `rx_nb_enable` and `rx_nb_enable_ex`.
RX_ANC_ENABLE	Not parsed as a client command in the current server.
RX_DSE_ENABLE	Not parsed as a client command in the current server.
LINE_OUT_RECORDER_START / SAVE / BREAK	Not parsed as client commands in the current server.
TX_FOOTSWITCH	No current Thetis sender was found for the official notification.
CHANNEL_COUNT	The official initialisation command is singular. Thetis and observed ExpertSDR3 traffic use `channels_count`.

4.3 Thetis Extension Commands Not In The Official 2.0 Text

The commands below are implemented by the current Thetis TCI server but are not listed as official commands in the uploaded Expert Electronics TCI 2.0 text.

Command	Type	Purpose	Example
CHANNELS_COUNT	Initialization reply	Thetis emits the plural initialisation message	`channels_count:2;`

Command	Type	Purpose	Example
		`channels_count:<count>;`. The official TCI 2.0 text names the command `channel_count`. Client code should accept the Thetis spelling when talking to Thetis.	
RX_ENABLE	Send / reply	Thetis receiver enable state. RX1 is effectively always available; RX2 can be enabled or disabled. This command appears in ExpertSDR3 style startup traffic but is not listed in the uploaded official 2.0 command list.	`rx_enable:1,true;`
TX_FILTER_BAND_EX	Send / reply	Thetis TX filter edge extension. Fields are low edge in Hz and high edge in Hz. Thetis normalises both values to non-negative order and enforces a minimum 100 Hz span.	`tx_filter_band_ex:100,2900;`
TX_FREQUENCY_EX	Notification	Extended TX frequency notification. Fields are TX frequency in Hz, Thetis band token, RX2 enabled flag, and TX on VFO B flag.	`tx_frequency_ex:7070468,b40m,false,false;`
RX_NR_ENABLE_EX	Send / reply	Extended RX noise reduction state. Fields are receiver index, enabled state, and NR selection index. Thetis accepts NR selection indexes `0` through `4`.	`rx_nr_enable_ex:0,true,2;`
RX_NB_ENABLE_EX	Send / reply	Extended RX noise blanker state. Fields are receiver index, enabled state, and NB selection index. Thetis accepts NB selection indexes `0` through `2`.	`rx_nb_enable_ex:0,true,1;`
RX_STEP_ATT_ENABLED_EX	Send / reply	Step attenuator enable state by receiver. The set path requires the Thetis setup form to be available.	`rx_step_att_enabled_ex:0,true;`
RX_STEP_ATT_EX	Send / reply	Step attenuator value by receiver. The value is a positive attenuation value in dB.	`rx_step_att_ex:0,10;`
RX_PREAMP_ATT_EX	Send / reply	Preamp attenuation extension by receiver. Thetis replies with a negative attenuation value derived from the current preamp mode. On set, positive values are ignored.	`rx_preamp_att_ex:0,-10;`
RX_CTUN_EX	Send / reply	Thetis CTUN state extension. Fields are	`rx_ctun_ex:0,true;`

Command	Type	Purpose	Example
		receiver index and enabled state.	
VFO_SYNC_EX	Send / reply	Thetis VFO sync state. Use no arguments to request the current state.	`vfo_sync_ex:true;`
VFO_SWAP_EX	Send	Immediately swaps the active VFO assignments in Thetis.	`vfo_swap_ex;`
FM_DEVIATION_EX	Send / reply	Thetis FM deviation extension. Current accepted set values are `2500` and `5000` Hz.	`fm_deviation_ex:0,2500;`
AGC_AUTO_EX	Send / reply	Thetis AGC auto enable extension. Fields are receiver index and enabled state.	`agc_auto_ex:0,true;`
RX_CHANNEL_SENSORS_EX	Notification	Extended channel sensor report sent with the standard channel sensor report during normal RX sensor subscriptions. Fields are receiver index, channel index, current signal dBm, average signal dBm, and peak bin signal dBm.	`rx_channel_sensors_ex:0,0,-71.5,-73.2,-69.8;`
RX_SENSORS_ENABLE_EX	Send	Compact per receiver/channel RX sensor subscription. Fields are receiver index, channel index, enabled state, and optional interval in milliseconds. The interval is clamped to the same `30` through `1000` ms range as normal sensors.	`rx_sensors_enable_ex:0,0,true,200;`
SR	Notification	Short compact RX sensor report emitted by `rx_sensors_enable_ex`. Fields are receiver index, channel index, signal dBm, average signal dBm, peak bin signal dBm, and noise floor dBm.	`sr:0,0,-71.5,-73.2,-69.8,-118.4;`
DISPLAY_START	Send / reply	Starts or reconfigures a Thetis display stream subscription. Current Thetis requires the 11-field form documented in section 3.3.	`display_start:0,1024,15,8192,4,0,2,250.0,true,-140.0,-20.0;`
DISPLAY_STOP	Send / reply	Stops the selected display stream subscription.	`display_stop:0;`
DISPLAY_PIXELS	Send / reply	Sets display line width. Thetis clamps values to `64` through `32768`.	`display_pixels:0,2048;`
DISPLAY_ZOOM	Send / reply	Sets zoom level and centre frequency. Thetis replies with the normalised zoom	`display_zoom:0,0.650000,7074000;`

Command	Type	Purpose	Example
		and follows with `display_pan`.	
DISPLAY_PAN	Send / reply	Sets pan position from `0.0` through `1.0`.	`display_pan:0,0.500000;`
DISPLAY_FRAMERATE	Send / reply	Sets requested display frame rate. Thetis clamps to `1` through `120` fps.	`display_framerate:0,25;`
DISPLAY_FFTSIZE	Send / reply	Sets the display FFT size. The value is normalised by the spectrum processor.	`display_fftsize:0,8192;`
DISPLAY_WINDOW	Send / reply	Sets display window type. Thetis clamps to `0` through `6`.	`display_window:0,4;`
DISPLAY_DETECTOR	Send / reply	Sets display detector type. Thetis clamps to `0` through `4`.	`display_detector:0,0;`
DISPLAY_AVERAGING	Send / reply	Sets display average mode. Thetis clamps to `0` through `3`.	`display_averaging:0,2;`
DISPLAY_AVERAGE_TIME	Send / reply	Sets display average time in milliseconds. Thetis clamps to its supported tau range.	`display_average_time:0,250.0;`
DISPLAY_ONE_HZ	Send / reply	Enables or disables one hertz normalisation for display output.	`display_one_hz:0,true;`
DISPLAY_DBM_RANGE	Send / reply	Sets display min and max dBm values used for quantisation and client scaling.	`display_dbm_range:0,-140.0,-20.0;`
DISPLAY_DELTA_MODE	Send / reply	Enables or disables delta capable display publication. Disabling it also disables block delta mode.	`display_delta_mode:0,true;`
DISPLAY_BLOCK_DELTA_MODE	Send / reply	Enables or disables block delta display publication. Enabling it also enables normal delta mode.	`display_block_delta_mode:0,true;`
DISPLAY_ACCURACY_MODE	Send / reply	Sets display compression tolerance. Values `0` through `4` are byte thresholds; `5` selects automatic tolerance.	`display_accuracy_mode:0,5;`
DISPLAY_ADAPTIVE_BLOCKS	Send / reply	Enables adaptive block size selection for block delta frames.	`display_adaptive_blocks:0,true;`
DISPLAY_KEYFRAME_INTERVAL	Send / reply	Sets the maximum number of delta frames between raw frames. `0` disables the forced keyframe interval.	`display_keyframe_interval:0,30;`
DISPLAY_QUANT_BITS	Send / reply	Sets raw display quantisation depth. Thetis clamps to `6`, `7`, or `8` bits.	`display_quant_bits:0,7;`

Command	Type	Purpose	Example
AUDIO_CODEC	Send / reply	Selects the negotiated audio codec family. Thetis accepts `pcm` and `opus`.	`audio_codec:opus;`
AUDIO_OPUS_APPLICATION	Send / reply	Sets Opus application mode. Accepted values are `audio`, `voip`, and `lowdelay`.	`audio_opus_application:audio;`
AUDIO_OPUS_SIGNAL	Send / reply	Sets Opus signal hint. Accepted values are `auto`, `voice`, and `music`.	`audio_opus_signal:voice;`
AUDIO_OPUS_BANDWIDTH	Send / reply	Sets Opus bandwidth hint. Accepted values are `auto`, `narrowband`, `mediumband`, `wideband`, `superwideband`, and `fullband`.	`audio_opus_bandwidth:fullband;`
AUDIO_OPUS_BITRATE	Send / reply	Sets Opus bitrate in bits per second. Thetis clamps to `6000` through `512000`.	`audio_opus_bitrate:64000;`
AUDIO_OPUS_VBR	Send / reply	Enables or disables Opus variable bitrate mode.	`audio_opus_vbr:true;`
AUDIO_OPUS_CVBR	Send / reply	Enables or disables constrained variable bitrate mode.	`audio_opus_cvbr:false;`
AUDIO_OPUS_COMPLEXITY	Send / reply	Sets Opus complexity. Thetis clamps to `0` through `10`.	`audio_opus_complexity:10;`
AUDIO_OPUS_FEC	Send / reply	Enables or disables Opus in band forward error correction.	`audio_opus_fec:true;`
AUDIO_OPUS_DTX	Send / reply	Enables or disables Opus DTX.	`audio_opus_dtx:false;`
AUDIO_OPUS_PACKET_LOSS	Send / reply	Sets expected packet loss percentage. Thetis clamps to `0` through `100`.	`audio_opus_packet_loss:5;`
AUDIO_OPUS_FRAME_MS	Send / reply	Sets Opus frame duration. Thetis normalises to `2.5`, `5`, `10`, `20`, `40`, or `60` ms and updates `audio_stream_samples`.	`audio_opus_frame_ms:20.0;`
AUDIO_RECONFIGURE_BEGIN	Send	Begins grouped audio reconfiguration. Use `audio_reconfigure_begin:;`. Thetis defers audio reset and queued TX audio clearing while the reconfiguration depth is above zero.	`audio_reconfigure_begin:;`
AUDIO_RECONFIGURE_END	Send	Ends grouped audio reconfiguration. Use `audio_reconfigure_end:;`. Deferred reset is applied when the nesting depth returns to zero.	`audio_reconfigure_end:;`

Command	Type	Purpose	Example
RX_SNB_ENABLE	Send / reply	Thetis spectral noise blanker state by receiver. Use `rx_snb_enable:<rx>;` to query or `rx_snb_enable:<rx>,<bool>;` to set. When authentication is enabled, this command requires `change_rx_dsp`.	`rx_snb_enable:0,true;`
DUPLEX	Send / reply	Thetis display duplex state. The implemented protocol command is `duplex`; there is no `display_duplex` command. Zero based receiver `0` can be queried with `duplex:0;` or set with `duplex:0,<bool>;`. Receiver `1` queries return `false` and set requests are ignored. When authentication is enabled, this command requires `change_display`.	`duplex:0,true;`
TUNE_TWOTONE	Send / reply	Thetis two tone tune state. Set requests use `tune_twotone:<rx>,<bool>;` and are accepted only when the supplied zero based receiver matches the active TX routing. The current one argument query path replies using receiver `0` regardless of the supplied argument, so clients should treat `tune_twotone:0;` as the only useful query form. When authentication is enabled, this command requires `tx_power`.	`tune_twotone:0,true;`
PS-A	Send / reply	Thetis PureSignal auto state. Set with `ps-a:<bool>;`. Although the handler contains a zero argument query branch, the current parser has no bare `ps-a;` case and `ps-a:;` is parsed as one empty argument, so there is no working client query command in the current source. The state is still sent by Thetis during initial state and when PureSignal auto changes. When authentication is enabled, this command requires `tx_settings`.	`ps-a:true;`
TX_PROFILE_EX	Send / reply	Selects or reports the active Thetis TX profile. Use the bare command `tx_profile_ex;` to request the current profile, or	`tx_profile_ex:dx;`

Command	Type	Purpose	Example
		`tx_profile_ex:<name>;` to set it.	
TX_PROFILES_EX	Send / reply	Returns the available Thetis TX profile names as a comma separated list.	`tx_profiles_ex;`
CALIBRATION_EX	Send / reply	Returns Thetis calibration values. Fields are receiver index, meter offset, display offset, XVTR gain offset, 6 m gain offset, and TX display offset.	`calibration_ex:0;`
RUN_CAT_EX	Send / reply	Executes one CAT command through Thetis and returns the CAT response wrapped in a TCI message. On error, Thetis replies with `?`.	`run_cat_ex:ZZFA;`
RT	Send / echo reply	Round trip timing helper. The client sends the bare command `rt;` and Thetis replies with the same bare command `rt;`. It has no arguments and is intended only as a lightweight echo for measuring total client to server to client latency.	`rt;`
SHUTDOWN_EX	Send	Requests a full Thetis shutdown.	`shutdown_ex;`
SPOT_SIMULATE_CLICK	Send	Triggers a simulated spot click in Thetis. Fields are callsign and frequency in Hz.	`spot_simulate_click:k1abc,14074000;`
SPOT arg5 JSON payload	Send	Thetis extension to the standard `spot` command. A fifth argument beginning with `[json]` can carry structured metadata.	`spot:k1abc,cw,7025000,4294967295,[json]{...};`

4.4 Current Parsed Command And Permission Map

The table below records the current permission mapping used by `clsTCIpermissions.cs`. It is included so that a client developer can implement authentication handling without reading the server source.

Permission	Commands covered	Notes
`radio_power`	`start`, `stop`	Power state controls.
`admin`	`shutdown_ex`, `run_cat_ex`	Administrative and CAT passthrough controls.
`change_mode`	`modulation`	Mode change.
`change_frequency`	`dds`, `if`, `vfo`, `vfo_swap_ex`, set forms of `digl_offset` and `digu_offset`	For `digl_offset` and `digu_offset`, the permission depends on whether arguments are supplied.
`change_rx_filter`	`rx_filter_band`	RX filter edge control.

Permission	Commands covered	Notes
view_spectrum	display_start, display_stop, iq_start, iq_stop	Stream start and stop controls.
change_display	display_zoom, display_pan, display_pixels, display_delta_mode, display_block_delta_mode, display_accuracy_mode, display_adaptive_blocks, display_keyframe_interval, display_quant_bits, display_framerate, display_fftsize, display_window, display_detector, display_averaging, display_average_time, display_one_hz, display_dbm_range, iq_samplerate, duplex	Display, IQ sample rate, and duplex display settings.
listen_rx_audio	audio_start, audio_stop, line_out_start, line_out_stop	RX audio and line out stream start and stop controls.
change_rx_audio	audio_samplerate, audio_codec, audio_reconfigure_begin, audio_reconfigure_end, audio_stream_sample_type, audio_stream_channels, audio_stream_samples, all audio_opus_* controls, tx_stream_audio_buffering, mute, volume, rx_mute, mon_volume, mon_enable, rx_volume, rx_balance, sql_enable, sql_level	Audio format and audio control settings.
change_rx_dsp	rx_nr_enable, rx_nr_enable_ex, rx_nb_enable, rx_nb_enable_ex, rx_snb_enable, rx_anf_enable, rx_bin_enable, rx_apf_enable, rx_nf_enable, agc_auto_ex, agc_mode, agc_gain	RX DSP, noise processing, and AGC controls.
change_rx_frontend	rx_enable, rx_channel_enable, rx_step_att_enabled_ex, rx_step_att_ex, rx_preamp_att_ex, vfo_sync_ex, fm_deviation_ex, rx_ctun_ex	RX frontend and receiver state controls.
tx_ptt	trx	Transmit PTT control.
tx_tune	tune	Tune control.
tx_power	drive, tune_drive, tune_twotone	TX drive and two tone tune controls.
tx_settings	split_enable, tx_filter_band_ex, tx_profile_ex, ps-a	TX profile, split, filter, and PureSignal auto settings.
tx_audio	binary_tx_audio	This applies to binary TX audio frames, not a normal text command.
cw_keyer	cw_macros_speed, cw_macros_delay, cw_keyer_speed, cw_macros_speed_up, cw_macros_speed_down, cw_macros, cw_terminal, cw_msg, keyer, cw_macros_stop, callsign_send	callsign_send is mapped even though the current server emits it as a notification.
spots	spot, spot_delete, spot_clear, spot_simulate_click	Spot management and simulated spot click handling.
view_status	rit_enable, xit_enable, rit_offset, xit_offset, lock, vfo_lock, set_in_focus, rx_sensors_enable, rx_sensors_enable_ex, tx_sensors_enable, tx_profiles_ex, calibration_ex, rt, query forms of digl_offset and digu_offset	Status, sensors, and low impact query controls.

SPOT JSON Payload In Arg5

Thetis accepts a structured JSON payload in the fifth `spot` argument by appending `[json]` immediately before the JSON object. The parser treats the JSON block as part of the same `spot` command even when it contains commas, quotes, or nested braces.

Command Format

```
spot:<callsign>,<mode>,<frequency_hz>,<argb>,[json]{...};
```

```
spot:g4abc,cw,7025000,4294967295,[json]{"Spotter":"m0xyz","Comment":"Loud on dipole","Heading":45,"Country":"England","UtcTime":"2025-06-25T16:45:30Z","TextColor":"#ffd700","Flag":"gb","FlagSpotter":"gb","Distance":128,"IsSWL":false,"SWLSecondsToLive":0};
```

JSON property	Type	Current Thetis behaviour
Spotter	string	Stored and displayed as the spotter callsign or source label.
Comment	string	Used as the spot comment text and replaces the plain arg5 free text field when JSON is present.
Heading	integer	Beam heading in degrees. Values outside `0` to `360` are treated as not present.
Continent	string	Accepted by the JSON schema, but the current Thetis add spot path does not populate the stored spot continent from this field.
Country	string	Stored as the spot country. If omitted, Thetis may derive country and flag information from the callsign prefix.
UtcTime	string	UTC timestamp in the form `yyyy-MM-ddTHH:mm:ssZ`. If parsing fails or the field is omitted, Thetis uses the current UTC time.
TextColor	string	Optional text colour override parsed by Thetis spot colour handling.
Flag	string	Optional flag asset code for the spotted station, using the ANSI two letter country code.
FlagSpotter	string	Optional flag asset code for the spotter, using the ANSI two letter country code.
Distance	integer	Optional distance in kilometres. When present, Thetis stores the supplied value with the spot.
IsSWL	boolean	Optional to mark the spot as SWL only behaviour for Thetis spot lifetime management.
SWLSecondsToLive	integer	Optional SWL expiry in seconds. Negative values are treated as `0`.

When JSON is used, Thetis deserialises arg5 into its internal `JsonSpotData` class and passes both the raw JSON text and parsed object into spot creation. If JSON deserialisation fails, the spot is ignored. The parser looks specifically for the marker `[json]{` inside the `spot` command. Without `[json]`, all content from arg5 onward is joined back together as free text.

5 Compatibility Notes

5.1 Reference Recommendation

Client authors should continue to use the official Expert Electronics TCI 2.0 document as the baseline reference. Where the published document conflicts with observed command usage, especially in the CW terminal family, Thetis interoperability notes in this supplement take precedence.

5.2 Display Client Guidance

- Use the 11-field `display_start` form. The older 6-field form is not accepted by the current Thetis implementation.
- Detect display binary frames by the leading bytes `84,68,1`, which represent `TD` version `1`.
- Keep a decoded previous frame for each receiver because delta, block delta, and use previous frame payloads depend on it.
- Ignore delta payloads until a raw payload has been received for that receiver.
- After any loss of binary display data, wait for the next raw payload before resuming delta decode for that receiver.

5.3 Validated Protocol 2.0 Deviations And Omissions

- `cw_terminal`: the published document shows a single boolean argument, but Thetis follows the receiver qualified form used by ExpertSDR 3 v1.1.7, for example `cw_terminal:0,true;`.
- `cw_macros_empty`: the published document shows a bare notification, but Thetis emits the receiver qualified form used by ExpertSDR 3 v1.1.7, for example `cw_macros_empty:0;`.
- Terminal TX hold: terminal mode remains in TX after queued CW completes and only drops on explicit terminal clear, stop, disconnect, or other hard abort conditions.

5.4 Routing And Receiver Topology

Several commands in Thetis map onto the VFO A / VFO B model rather than a strict one command per receiver abstraction. This is most visible in `lock`, `vfo_lock`, and `keyer` when RX2 is enabled.

5.5 CW Mode Preconditions

Thetis does not silently force a non CW mode into CW for TCI CW operations. TCI CW activity is expected to operate only when the selected TX side is already in `cwl` or `cwu`.

5.6 Implementation Notes

- The duplex command is `duplex`. There is no `display_duplex` text command.
- `duplex` requires `change_display`, `rx_snb_enable` requires `change_rx_dsp`, `tune_twotone` requires `tx_power`, and `ps-a` requires `tx_settings` when authentication is enabled.
- `audio_reconfigure_begin` and `audio_reconfigure_end` are handled in the colon command branch. Use `audio_reconfigure_begin:;` and `audio_reconfigure_end:;`.
- `ps-a` can be set, and is sent by the server, but the current parser does not expose a working client query form.

5.7 Client Guidance

- Use the official Expert Electronics 2.0 document for generic command grammar and message sequencing, but prefer the validated ExpertSDR 3 v1.1.7 CW terminal forms when they differ from the PDF.
- Use this supplement for Thetis routing, AGC mapping, CW/CWX behaviour, sensor negotiation, compact `sr` reports, stream negotiation, VFO sync/swap helpers, display compression, round trip `rt` echo checks, official command deviations, and Thetis extension messages.
- Prefer `tx_frequency_ex` rather than the retired `tx_frequency_thetis` extension when you need Thetis band and TX routing context.
- Treat CW ownership as exclusive in Thetis.
- Do not assume that all TCI receivers in Thetis map directly to independent hardware or DSP chains.

End of document.